Evaluating Reinforcement Learning for Emergency SAGIN Connectivity: A Tutorial Case Study of Qusin, Nablus

Mohammed Allan Yadak
Department of Computer Science, An-Najah National University
Registration No. 12217817
Email: hammodallan9@gmail.com

***Abstract*—This paper is a step-by-step tutorial on taking an open-source Space–Air–Ground Integrated Network (SAGIN) toolkit from a public repository to a reproducible and honest evaluation on a concrete local problem: an emergency back-up network for Qusin and five neighbouring villages near Nablus, assuming the terrestrial network is down. We reuse a tabular Q-learning toolkit for joint user association and power control, place users around real villages in proportion to population, and show that a simple control experiment exposes an unrealistic assumption in the toolkit's radio model. We then extend the model with spot beams, frequency reuse, per-HAPS capacity and a directional backhaul, and compare Q-learning with seven baselines over 30 random topologies using paired bootstrap confidence intervals. Q-learning clearly beats naive baselines, but capacity-aware heuristics match or beat it on outage, and a minimum-power heuristic is far more energy-efficient; antenna directivity matters more than the choice of algorithm. All steps, parameters and code are documented for reproduction.**

***Index Terms*—SAGIN, HAPS, LEO, reinforcement learning, user association, power control, emergency connectivity, reproducibility.**

I. Introduction

When the terrestrial mobile network of a rural area stops working—because of a power cut, damaged towers or any other emergency—people lose the ability to call for help or receive information. A SAGIN can act as a fall-back: low-Earth-orbit (LEO) satellites provide the backhaul, high-altitude platform stations (HAPS) at about 20 km act as "flying base stations", and terminals on the ground connect to a HAPS. The practical question is how to assign ground users to HAPS and how much power each transmitter should use so that nobody is left without service. Joint user association and beamforming in satellite–HAPS–ground networks has been studied under payload and backhaul constraints in [1], and machine-learning-based user scheduling in such networks in [2].

This paper has a different purpose. It is a hands-on tutorial, in the spirit of [3], that follows a workflow any student can repeat on a laptop without a GPU or a paid API: (1) choose a repository from the GitHub topic `sagin`; (2) understand it (why, what, where, when, how); (3) find a problem worth solving that is connected to real life; (4) run the tool and apply it to that problem; (5) evaluate against fair baselines; (6) report reproducibly. Sections II–VI follow these steps.

The contributions are: (i) a population-weighted emergency scenario for Qusin and five neighbouring villages built from public data; (ii) a control experiment showing that the toolkit's default radio model favours a trivial solution, and a more realistic model (spot beams, frequency reuse, capacity limit, directional backhaul) added *without modifying the toolkit*; (iii) a comparison with seven baselines over 30 random topologies with paired bootstrap confidence intervals; and (iv) complete, reproducible code and results.

Main findings. With the original radio model, putting *all* users on one HAPS is optimal, which is physically unrealistic. After making the model more realistic, Q-learning still clearly beats naive baselines, but a capacity-aware greedy heuristic has lower outage (0.4% vs. 10.7% at 45 users), a minimum-power variant of that heuristic needs far less transmit power, and antenna directivity matters more than the algorithm.

This is a simulation study under stated assumptions, not a deployment proposal or a measurement of any existing network. The selected repository itself states that it claims no research novelty [4]; we adopt the same stance.

II. System Description

A. SAGIN Layers and Link Model

A SAGIN has three layers: LEO satellites (300 km altitude, the toolkit's default), HAPS platforms at about 20 km, and ground users (GUs). Two links matter: the *backhaul* (tier 1, LEO→HAPS) and the *access* link (tier 2, HAPS→GU). Each receiver chooses exactly one transmitter (the *association*) and each transmitter chooses one power level from a discrete set (the *power control*). Receiver j served by transmitter a_j experiences

$$\mathrm{SINR}_j = \frac{P_{a_j}\, g_{a_j,j}}{\sum_{m \neq a_j,\ m \text{ active}} P_m\, g_{m,j} + N_0}, \tag{1}$$

where P_m is the transmit power, $g_{m,j}$ the channel gain (free-space path loss, antenna gain and Rayleigh fading) and N_0 the noise power. The spectral efficiency is $\log_2(1+\mathrm{SINR}_j)$ in $\mathrm{bit/s/Hz}$; multiplied by the 10 MHz bandwidth it gives $\mathrm{Mbit/s}$. Raising one's own power helps oneself but hurts every receiver sharing the band, which makes power control a genuine optimisation problem.

B. Q-Learning in the Toolkit

Q-learning [5] learns by trial and error a table $Q(s,a)$ of how good each action a is in each state s. In the toolkit

Table I: The two repositories under the `sagin` topic (4 Oct. 2026).

	Q-learning toolkit	**sagin-agent-bench**
Idea	Tabular Q-learning for association and power in LEO–HAPS–ground networks	Benchmark for LLM agents in a space-network setting
Stars	7	0
Laptop-only?	Yes (Python, NumPy)	Needs an LLM API
Tests, baselines	5 tests, 3 baselines	Not assessed
Last activity	14 Jun. 2026	21 Jul. 2026

Table II: The five questions about the selected toolkit.

Question	**Answer**
Why	A correct, reproducible rewrite of an older MATLAB project whose README lists several fixed errors (e.g., path loss used as a gain, kilometre/metre mix-ups, an interference model where maximum power was always optimal); a teaching and baseline reference with no novelty claim.
What	Python + NumPy: independent tabular Q-learning agents for association and power in a two-tier SAGIN, three baselines, five unit tests.
Where	Coverage extension, disaster and emergency communications, teaching; not a production planner (no mobility, terrain or traffic model).
When	Quick, transparent experiments (seconds to minutes) and as a baseline for new methods.
How	`git clone`, `pip install -r requirements.txt`, `python tests.py`, `python experiment.py`.

every decision-maker (each user choosing a HAPS, each HAPS choosing a power, and so on) is an independent learner whose state is its own previous action. Association agents are rewarded with their own link rate and power agents with the tier sum-rate; exploration follows an ϵ-greedy policy with linear decay.

C. Steps 1–2: Selecting and Understanding the Repository

The topic page https://github.com/topics/sagin listed only *two* repositories on 4 October 2026 (Table I), so popularity could not be the only criterion. We asked practical questions any student should ask before investing time, and selected the Q-learning toolkit [4]: it is the more starred, it runs on a laptop in seconds to minutes, it ships tests and baselines, and its quantities (throughput, power, association) translate into service-quality questions. The other repository needs an LLM API, which makes it harder to reproduce and harder to tie to a village-scale problem.

Before using a tool one should be able to answer five questions about it (Table II). The repository is maintained by the GitHub account `Lewis-panda`; its commit history shows an older MATLAB project and a clean Python rewrite dated 14 June 2026.

D. Step 3: A Problem Worth Solving

We chose a problem from our own surroundings that the tool can help to reason about: *if the terrestrial network around Qusin (about 7.7 km west of Nablus) and its neighbouring villages were unavailable, how should a small SAGIN fall-back—four HAPS backed by three LEO satellites—serve the people there, and how much does the choice of algorithm matter?* A fall-back plan is useful only if it serves nearly everybody at a usable rate, does not waste scarce transmit energy, and does not depend on a single point of failure.

Table III: Scenario data (about 16 500 people in an area of roughly 5×7 km).

Village	**Lat. (N)**	**Lon. (E)**	**Pop. 2017**	**σ (km)**
Qusin	32.23861	35.18333	2 251	0.6
Deir Sharaf	32.25417	35.18806	2 949	0.6
Beit Iba	32.23611	35.20972	4 079	0.7
Sarra	32.21000	35.19083	3 384	0.6
Zawata	32.24583	35.22611	2 537	0.6
Beit Wazan	32.23139	35.21583	1 312	0.5

Instead of the toolkit's uniform random layout, users are placed around six real settlements in proportion to population (Table III). Coordinates and 2017 census populations come from public village pages that cite the Palestinian Central Bureau of Statistics [6]; the spread σ is our assumption. Latitude/longitude are converted to local kilometres with $x = (\lambda - \lambda_0)\,111.32\cos\varphi_0$ and $y = (\varphi - \varphi_0)\,110.57$. Each simulated user is a *representative* user standing for a group of real people; we use 30 users (moderate load) and 45 users (busy load) and sweep from 15 to 60. Four HAPS at 20 km are spread over the villages' bounding box with 0.5 km station-keeping jitter, and three LEO satellites are placed uniformly within ± 150 km.

We judge methods by metrics that matter to the people served, not only the sum-rate: *outage* (percentage of users whose average end-to-end rate is below $1\,\mathrm{Mbit/s}$, our assumed minimum useful rate), the *5th-percentile user rate*, the *total transmit power* of active transmitters, the percentage of *blocked* users, and the sum-rate. The end-to-end rate of a user is the *minimum* of its access rate and its share of the HAPS backhaul.

III. Running the Toolkit and a Red Flag

A. Step 4a: Setup, Tests and Wrapping the Toolkit

The toolkit is installed and verified with a few commands (Windows or Linux):

```
git clone https://github.com/Lewis-panda/Optimizing-SAGIN-
    Association-Link-Power-Using-Reinforcement-Learning.
    git sagin
cd sagin
pip install -r requirements.txt # numpy, matplotlib
python tests.py # 5 sanity tests
python experiment.py --quick # smoke run
```

All five tests pass, and the quick run shows Q-learning 21.6% above the best baseline in sum-rate (the README reports +95.6% for its full default scenario [4]). We never edit the repository's files; new files placed beside them import its functions. `qusin_scenario.py` defines the villages (Table III), draws a village for each user with probability proportional to population, and adds Gaussian scatter with that village's σ.

B. Step 4b: The First Experiment

We ran Q-learning (20 000 episodes, 10 random topologies) with the toolkit's baselines and one extra control, `SingleHAPS`, which puts *all* users on the single best HAPS.

Table IV: First model (toolkit's radio model, 30 users).

Method	Outage (%)	P5 rate	Power (W)	Active HAPS
Q-learning	14.0	0.055	18.6	1.5
MaxPower+Greedy	51.0	0.035	51.0	4
BestUniform+Greedy	51.7	0.035	51.0	4
SingleHAPS (control)	10.0	0.068	21.0	1

At first sight this looks like a success: Q-learning reduces outage from about 51% to 14% (Table IV). The control tells a different story: putting everybody on one HAPS is better still. Fig. 1 shows why: Q-learning *switched off* most HAPS (1.5 of 4 active on average). It had not learned load balancing; it had rediscovered that, in this model, the best strategy is to have only one transmitter.

C. Diagnosis

We first tested the natural hypothesis that the satellite backhaul was the bottleneck. It was not: it limited only 2–6% of users. The real cause is the radio model. All HAPS reuse the same band with the same antenna gain in every direction, and they are at 20 km altitude while the area is only about 5 km wide, so from any user the serving HAPS and the others are at almost the same distance and gain. Then $\mathrm{SINR} \approx 1/(K-1)$ for K active HAPS: adding transmitters hurts and switching them off helps. Real HAPS use narrow spot beams, separate sub-bands and have limited capacity, none of which is in the toolkit's model.

Lesson 1: always ask what trivial strategy could achieve the same result before believing a gain.

IV. Improved Model and Fair Baselines

A. Step 4c: Making the Radio Model Realistic

We built `sagin_v2.py`, which reuses the toolkit's agents, fading and path loss but replaces the access and backhaul rate computations. There are four changes.

Spot beams. A HAPS points a beam at the user it serves; the desired link gets the peak gain G (35 dBi by default). Interference reaches another user through the beam pattern, which falls with the angle θ between the beam direction and the victim user as seen from the HAPS:

$$A(\theta) = \min\Big(12\big(\theta/\theta_{3\mathrm{dB}}\big)^2,\ \mathrm{SLA}\Big)\ \mathrm{dB}, \tag{2}$$

$$\theta_{3\mathrm{dB}} = \sqrt{30000/G_{\mathrm{lin}}}\ \mathrm{deg}, \tag{3}$$

with side-lobe cap $\mathrm{SLA} = 25$ dB. This parabolic main-lobe form is of the same type as the antenna element patterns in 3GPP TR 38.901 [7]; we use it as a simple, transparent approximation. For $G = 35$ dBi, $\theta_{3\mathrm{dB}} \approx 3.1^\circ$, about a 1 km footprint from 20 km altitude (Fig. 2). Beams are scheduled round-robin, so the interference a user sees from HAPS m is the *average* over m's served users.

Frequency reuse. The band is split into F sub-bands (default $F = 2$). Neighbouring HAPS (alternating in east–west order) use different sub-bands, so interference comes only from HAPS in the same sub-band, but each owns only B/F of the spectrum.

Table V: Parameters of the improved model.

Parameter	Value	Origin
Carrier / bandwidth	2 GHz / 10 MHz	toolkit
LEO / HAPS altitude	300 km / 20 km	toolkit
Noise figure; power levels	7 dB; 10 levels, 10–40 dBm	toolkit
HAPS beam peak gain	35 dBi	assumption
Backhaul dish gain	40 dBi	toolkit
Side-lobe cap; sub-bands F	25 dB; 2	assumption
Capacity $k_{\max}$; outage threshold	12 users; 1 Mbit/s	assumption
Q-learning	α=0.1, γ=0.9, $\epsilon: 1 \to 0.01$ over 60% of 20 000 episodes	toolkit

Table VI: Compared methods.

Method	Description
Q-Learning	Independent tabular Q-learning agents (association and power, both tiers).
Random	Random associations and powers.
MaxPower+Greedy	Strongest-signal association, maximum power.
BestUniform+Greedy	Strongest-signal association; one common power maximising sum-rate.
`GreedyLB`	Strongest users first; each takes its best HAPS with room; best common power.
`GreedyLB+LEO`	`GreedyLB` plus balanced HAPS→LEO association (at most $\lceil H/L \rceil$ HAPS per LEO).
`GreedyLB+LEO (minP)`	As `GreedyLB+LEO` with the *smallest* common power keeping topology outage $\leq 1\%$.
SingleHAPS	Control: all users on the best HAPS (capacity respected, so most are blocked).

Capacity. A HAPS serves at most $k_{\max} = 12$ users; if more are associated, only the 12 strongest are admitted and the rest are *blocked* (rate 0), a simplified form of the payload connectivity constraint studied in [1]. Q-learning feels the penalty through its reward.

Directional backhaul. This fourth change was discovered during evaluation (Section IV-B). The toolkit lets satellites interfere at full strength, so using two LEO is penalised. Each HAPS actually points a high-gain dish (40 dBi) at its serving LEO, so we apply the pattern (2) to the angular separation of the LEOs seen from each HAPS.

For user j served by HAPS $m = a_j$ and admitted there, with $\mathcal{A}_m$ the set of users admitted at m and $f(\cdot)$ the sub-band,

$$R_j = \frac{1}{F\,|\mathcal{A}_m|}\log_2\big(1+\mathrm{SINR}_j\big), \tag{4}$$

$$\mathrm{SINR}_j = \frac{P_m g_{m,j}}{\sum_{m' \neq m,\, f(m')=f(m)} P_{m'} g_{m',j}\,\bar{\rho}_{m',j} + N_0/F}, \tag{5}$$

with $\bar{\rho}_{m',j} = |\mathcal{A}_{m'}|^{-1}\sum_{i\in\mathcal{A}_{m'}} 10^{-A(\theta_{ij}^{m'})/10}$. The end-to-end rate is $r_j = \min\{R_j,\, R_m^{\mathrm{bh}}/|\mathcal{A}_m|\}$, with R_m^{bh} the backhaul rate of HAPS m computed in the same way; blocked users have $r_j = 0$. Table V lists the parameters; everything marked "assumption" is varied in the sweeps of Section V.

B. Step 5: Fair Baselines

A result is only as meaningful as the baselines it is compared with. We use eight methods (Table VI); the last three were added *because of* problems we found.

Why `GreedyLB`. Strongest-signal association overloads the best HAPS, and with a capacity limit the extras are blocked;

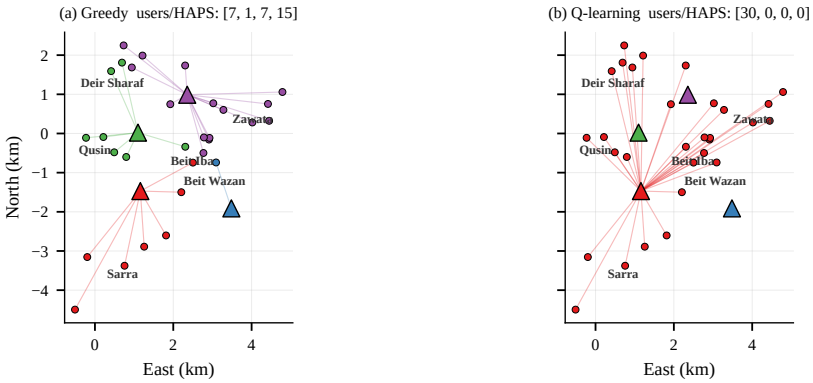


Fig. 1: First model, one topology. (a) Greedy association uses all four HAPS. (b) Q-learning puts all 30 users on one HAPS.

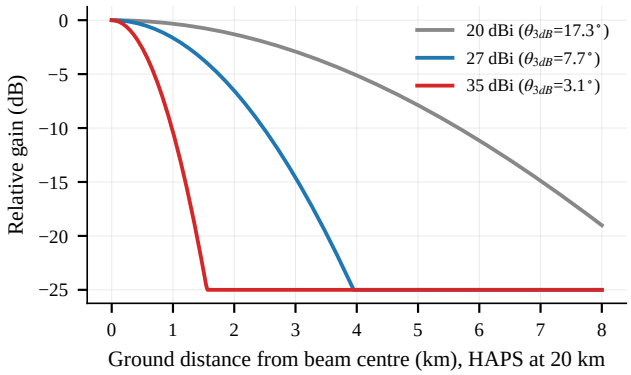


Fig. 2: Beam pattern for three peak gains. A 20 dBi beam is almost omnidirectional over the area; a 35 dBi beam separates users 1–2 km apart.

`GreedyLB` is the obvious engineering fix and costs a few lines of code.

Why `GreedyLB+LEO`. With strongest-signal LEO association the HAPS tended to choose the *same* satellite, so its band was divided four ways. In a diagnostic run on six topologies all four HAPS picked the same satellite in five of them (three of four in the sixth), and 21 of 21 outage users under `GreedyLB` were limited by the backhaul rather than the access link. The repository's README likewise observes that most of the learner's advantage comes from tier-1 load balancing [4], so a baseline without balanced LEO association is unfairly weak.

Why `minP`. Choosing the common power that maximises sum-rate always ends at the maximum, which wastes energy, so comparing a learned policy's power with it is unfair. `minP` uses the smallest power that meets the outage target; on average it needs 7.3 W (30 users) and 15.1 W (45 users) in total, against 28.7 W and 30.0 W for Q-learning.

Statistics. Results vary strongly across random topologies: the same configuration gave 2.0% and 3.7% Q-learning outage in two 10-topology runs with different seeds. For the headline comparison we use **30 topologies per load** and report paired differences (Q-learning minus baseline on the same topology) with 95% bootstrap confidence intervals (5 000 resamples). Error bars in figures are standard errors across topologies.

Lesson 2: strong, fair baselines and paired statistics turn a plot into evidence; the two baselines that changed our conclusions each took only a few lines of code.

V. Performance Analysis

A. Step 6: Main Comparison

Table VII and Fig. 3 show 30 topologies at each of two loads (3 LEO, 4 HAPS, 35 dBi beams, $F = 2$, $k_{\max} = 12$).

Three observations follow. (1) *At 30 users the load is easy:* even random association and power give everybody the 1 Mbit/s threshold on average; the capacity-blind strategies show about 5% outage and the one-HAPS control fails badly. (2) *At 45 users the methods separate:* strongest-signal strategies block 17% of users, `GreedyLB` has 0.4% outage and Q-learning 10.7%. (3) *Energy differs strongly:* about 29–30 W for Q-learning, 50–60 W for the rate-maximising baselines and only 7–15 W for `minP`.

B. Paired Comparison with Confidence Intervals

Table VIII reports differences (Q-learning minus baseline) on identical topologies. For outage and power negative values favour Q-learning; for rates positive values do.

Versus the naive baselines (BestUniform+Greedy, equivalently MaxPower+Greedy), Q-learning is clearly better at 45 users: outage -6.4 pp (CI excludes zero), 5th-percentile rate $+0.85$ Mbit/s and about 20 W less power. *Versus capacity-aware heuristics* it is not better on outage: at 45 users it is worse than `GreedyLB` by 10.3 pp (CI excludes zero) and worse on the 5th-percentile rate against all three heuristics; at 30 users it is about 2 pp worse than `GreedyLB` and `GreedyLB+LEO`. *Versus `GreedyLB+LEO`* it has equal sum-rate and uses about 30 W less. *Versus `minP`* the outage difference is not distinguishable from zero, but `minP` uses 14.9–21.4 W less power. We did not investigate why `GreedyLB+LEO` has more outage than `GreedyLB` at 45 users (4.6% vs. 0.4%) although its sum-rate is higher; choosing power to maximise sum-rate does not minimise outage, but we have not tested this explanation.

Table VII: Main results, mean over 30 random topologies per load. Sum = sum of end-to-end user rates; P5 = 5th-percentile user rate; Out. = outage; Blk. = blocked users.

Method	30 users					45 users				
	Sum (Mb/s)	P5 (Mb/s)	Out. (%)	Power (W)	Blk. (%)	Sum (Mb/s)	P5 (Mb/s)	Out. (%)	Power (W)	Blk. (%)
Q-Learning	121.8	2.36	2.4	28.7	0.2	113.6	0.96	10.7	30.0	1.9
Random	54.9	1.47	0.0	11.7	0.5	53.3	0.96	14.5	11.8	7.5
MaxPower+Greedy	81.3	1.15	4.9	48.7	4.6	85.1	0.11	17.1	50.0	16.7
BestUniform+Greedy	81.4	1.15	5.1	48.7	4.6	85.1	0.11	17.0	50.0	16.7
`GreedyLB`	83.5	1.71	0.4	49.7	0.0	83.9	1.48	0.4	50.3	0.0
`GreedyLB+LEO`	121.1	2.29	0.2	59.7	0.0	113.8	1.73	4.6	60.3	0.0
`GreedyLB+LEO (minP)`	66.5	1.27	2.1	7.3	0.0	79.5	1.33	4.8	15.1	0.0
SingleHAPS	22.3	0.00	60.0	20.0	60.0	22.7	0.00	73.3	20.3	73.3

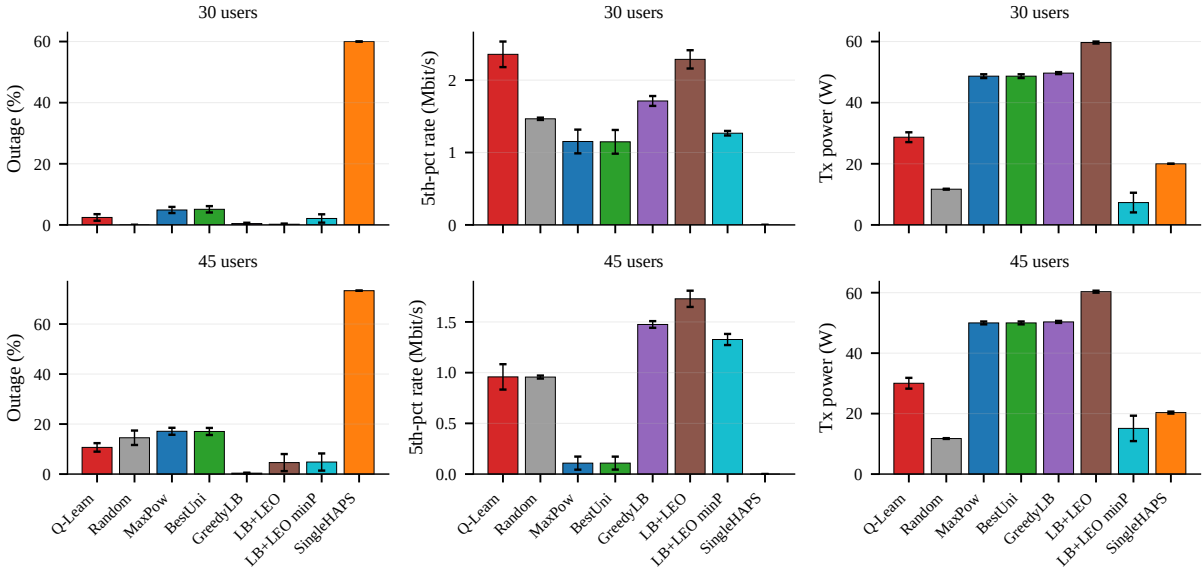


Fig. 3: Outage, worst-case user rate and transmit power at 30 users (top) and 45 users (bottom); error bars are standard errors over 30 topologies.

Table VIII: Paired differences, Q-learning minus baseline: mean [95% bootstrap CI] over 30 topologies (pp = percentage points).

Load	Versus	Δ outage (pp)	Δ P5 (Mb/s)	Δ power (W)	Δ sum (Mb/s)
30	`GreedyLB+LEO (minP)`	0.33 [−3.33, 3.22]	1.09 [0.73, 1.45]	21.4 [13.8, 27.6]	55.3 [47.4, 62.4]
30	`GreedyLB+LEO`	2.22 [0.22, 4.56]	0.07 [−0.28, 0.43]	−31.0 [−34.1, −27.9]	0.7 [−3.5, 4.6]
30	`GreedyLB`	2.00 [0.22, 4.22]	0.64 [0.30, 0.99]	−21.0 [−24.2, −18.0]	38.3 [35.1, 41.3]
30	BestUniform+Greedy	−2.67 [−5.56, 0.44]	1.21 [0.77, 1.65]	−20.0 [−23.3, −16.7]	40.4 [37.0, 43.8]
45	`GreedyLB+LEO (minP)`	5.85 [−3.04, 12.59]	−0.37 [−0.66, −0.08]	14.9 [4.9, 23.8]	34.2 [24.1, 43.4]
45	`GreedyLB+LEO`	6.07 [−2.74, 12.67]	−0.77 [−1.06, −0.45]	−30.3 [−33.8, −26.7]	−0.2 [−7.7, 8.2]
45	`GreedyLB`	10.30 [7.11, 13.78]	−0.52 [−0.78, −0.25]	−20.3 [−24.0, −16.6]	29.7 [23.8, 34.8]
45	BestUniform+Greedy	−6.37 [−10.52, −2.22]	0.85 [0.62, 1.09]	−20.0 [−23.6, −16.0]	28.5 [22.5, 33.6]

C. Behaviour of the Learned Policy

Fig. 4 shows one topology. Strongest-signal association overloads one HAPS (15 users for a limit of 12, so 3 are blocked); `GreedyLB` and Q-learning respect the limit. The associations learned by Q-learning are less intuitive: some users connect to a more distant HAPS, plausibly to keep interference in the shared sub-band low. Fig. 5 shows convergence: the training reward rises from about 21 to 33.5 and stabilises when exploration ends at episode 12 000.

D. Sensitivity to Load, Frequency Reuse and Beam Gain

Fig. 6 and Table IX vary one factor at a time (10 topologies per point, so the curves are indicative rather than conclusive).

Load. Up to 30 users every sensible method serves nearly everybody; at 60 users demand exceeds the capacity of four HAPS (48 slots), so even the best methods show about 20% outage and Q-learning is worse (32%). *Frequency reuse.* More sub-bands help Q-learning (8.3%→0.0%) while barely changing the heuristics, whose outage is already near zero; Q-learning also uses more power (25.9→35.6 W). *Beam gain* is the dominant factor: at 20 dBi every method has 20–

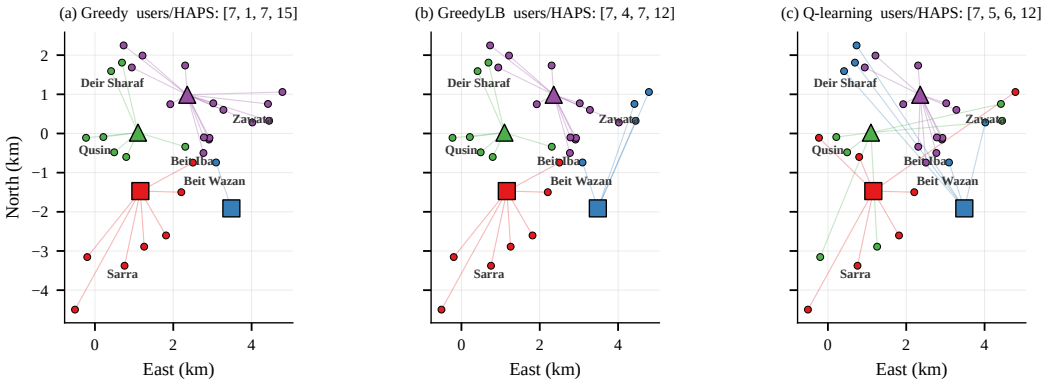


Fig. 4: Associations in the improved model for one topology. Colours identify the serving HAPS; marker shape identifies the sub-band.

Table IX: Outage (%) in the sensitivity sweeps (10 topologies per point; baseline point: 30 users, $F = 2$, 35 dBi).

	Users				Sub-bands F			Beam gain (dBi)		
Method	15	30	45	60	1	2	4	20	27	35
Q-Learning	0.0	3.7	4.7	32.0	8.3	3.7	0.0	19.7	9.3	3.7
`GreedyLB`	0.0	0.0	0.2	20.3	0.0	0.0	0.0	31.3	3.0	0.0
`GreedyLB+LEO`	0.0	0.0	0.2	20.2	0.0	0.0	0.0	28.3	1.7	0.0
`GreedyLB+LEO (minP)`	3.3	6.7	0.7	20.3	4.0	6.7	3.3	25.7	5.3	6.7
BestUniform+Greedy	0.0	2.0	13.8	26.8	2.0	2.0	2.0	29.3	4.7	2.0
SingleHAPS	20.0	60.0	73.3	80.0	–	–	–	–	–	–

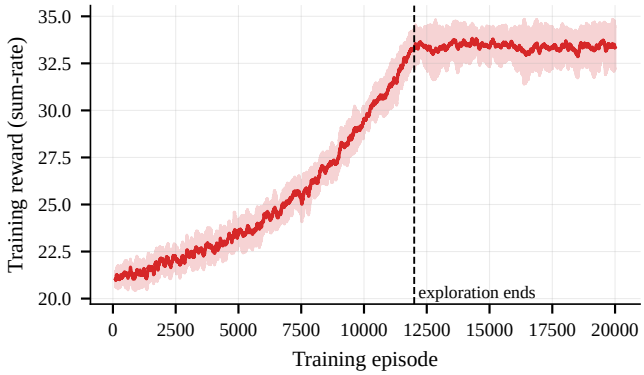


Fig. 5: Q-learning training reward (mean ± s.d. over 10 topologies, moving average).

31% outage and Q-learning is best in that regime (19.7% vs. 25.7–31.3%), while at 27 and 35 dBi `GreedyLB` and `GreedyLB+LEO` overtake it.

VI. Discussion and Reproducibility

A. What We Learned

(1) *Hardware first, algorithm second:* antenna directivity changes outage from roughly 30% to near zero, more than any choice of algorithm. (2) *Respect capacity:* letting a HAPS take more users than it can serve is the largest avoidable loss (17% blocked at 45 users). (3) *Balance the backhaul:* assigning HAPS to different satellites doubles the backhaul share compared with all HAPS using one satellite. (4) *Power can be small:* with good beams a uniform power of 7–15 W in total (`minP`) keeps average outage in single digits at these loads. Q-learning is a reasonable, easy-to-run baseline that clearly beats naive strategies and can win when the radio environment is poor (20 dBi); under good conditions simple capacity-aware heuristics are as good or better on outage and use less power. Because the toolkit's reward is the sum-rate, it does not target outage directly; rewarding outage reduction is a natural next experiment, but we did not run it and make no claim about its outcome.

B. Limitations

Beam gain, side-lobe cap, sub-band count, capacity, the 1 Mbit/s threshold and the user spread are assumptions (the sweeps use only 10 topologies). Thirty and 45 users are abstractions of about 16 500 residents. The physics is simplified: no mobility, rain, terrain or bursty traffic, equal sharing with round-robin beams, LEO altitude of 300 km (low for many real constellations), and only independent tabular Q-learning was tested. The study says nothing about the actual condition of any network in the area; it is a thought experiment on a realistic layout.

C. Pitfalls and Reproducibility

Three pitfalls cost us time and are worth avoiding: trusting a result without a control; weak baselines (greedy satellite selection made every baseline look worse than it should); and too few topologies (10-topology numbers moved noticeably between seeds). All code and results are available at https://github.com/hamoodallan9-sudo/wireless-networks-projects; the scripts are placed beside the unmodified toolkit repository (`qusin_scenario.py`, `sagin_v2.py`, `paired_stats.py` and the figure scripts):

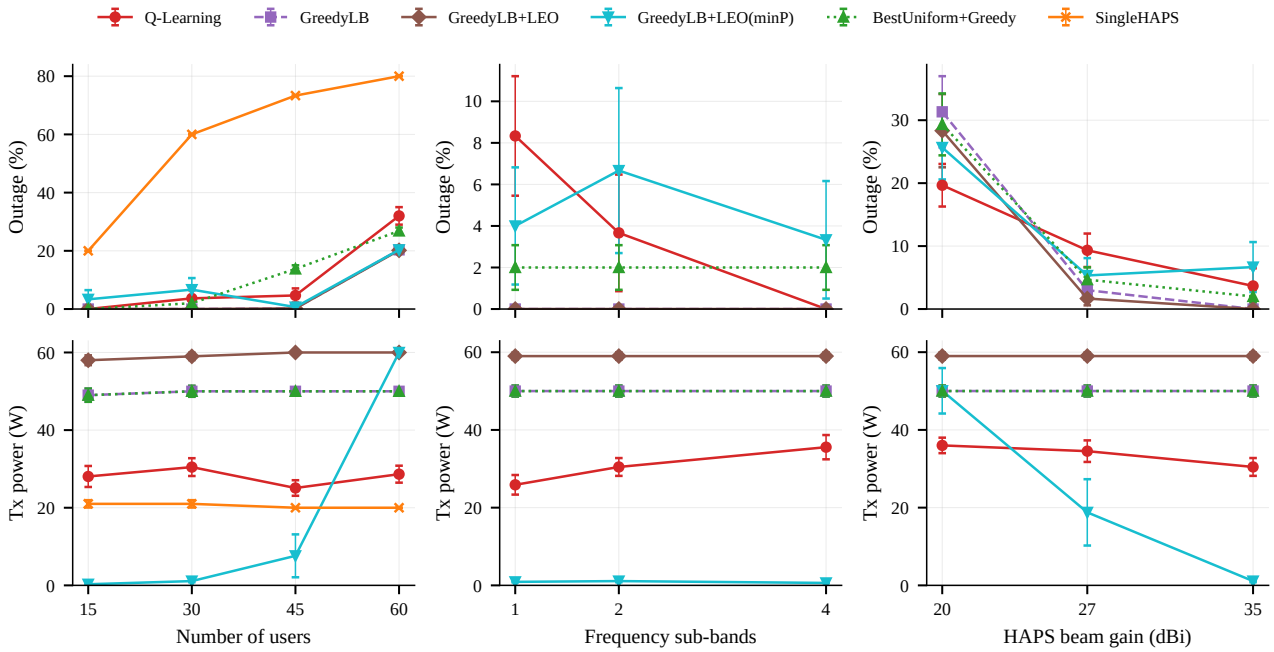


Fig. 6: Outage (top) and transmit power (bottom) versus number of users (left), sub-bands (middle) and HAPS beam gain (right); error bars are standard errors over 10 topologies.

```
python tests.py # toolkit tests
python sagin_v2.py # main run + sweeps, 10-15 min
python paired_stats.py # 30 topologies, ~10 min
python make_ieee_figures.py # figures (PDF)
```

Seeds are fixed (`--seed` for the main run, $777 + n$ for the paired runs), so reruns give identical numbers on the same library versions; no GPU is needed.

VII. Conclusions

We followed a complete workflow: pick a repository from a GitHub topic, understand it, tie it to a local problem—an emergency fall-back network for Qusin and its neighbouring villages—run it, and evaluate honestly. The most valuable outcome is a method rather than a number. A control experiment showed that the toolkit's default radio model made "switch everything off but one HAPS" optimal; a more realistic model with spot beams, frequency reuse, capacity and directional backhaul changed the picture; and fair baselines with paired statistics showed that Q-learning beats naive strategies but not simple, well-designed heuristics on outage, and that antenna directivity matters more than either.

Acknowledgment

Code and text drafting in this project were carried out with the assistance of an AI model (Claude, by Anthropic); the author reviewed the work and is responsible for its content. The toolkit used is the work of its GitHub maintainer, `Lewis-panda`.

References

[1] S. Liu, H. Dahrouj, and M.-S. Alouini, "Joint user association and beamforming in integrated satellite-HAPS-ground networks," arXiv preprint arXiv:2204.13257, 2022.

[2] S. Liu and H. Dahrouj, "Machine learning-based user scheduling in integrated satellite-HAPS-ground networks," arXiv preprint arXiv:2205.13958, 2022.

[3] S. Iqbal and J. M. Hamamreh, "A comprehensive tutorial on how to practically build and deploy 5G networks using open-source software and general-purpose, off-the-shelf hardware," *RS Open Journal on Innovative Communication Technologies*, vol. 2, no. 6, 2021.

[4] Lewis-panda, "Optimizing-SAGIN-association-link-power-using-reinforcement-learning," GitHub repository, 2026, https://github.com/Lewis-panda/Optimizing-SAGIN-Association-Link-Power-Using-Reinforcement-Learning, accessed Oct. 2026.

[5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[6] Palestinian Central Bureau of Statistics, "2017 population, housing and establishment census, preliminary results," as reported on the public Wikipedia pages for Qusin, Deir Sharaf, Beit Iba, Sarra, Zawata and Beit Wazan, accessed Oct. 2026.

[7] 3GPP, "Study on channel model for frequencies from 0.5 to 100 GHz," 3rd Generation Partnership Project, Technical Report TR 38.901.